

Robot Learning

Lecture 9 Sim-to-Real Transfer

Outline

- Opportunities of learning in simulation
- Sim-to-real gap minimization

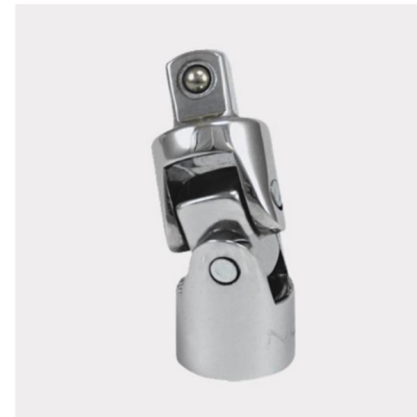
Sim-to-real policy transfer



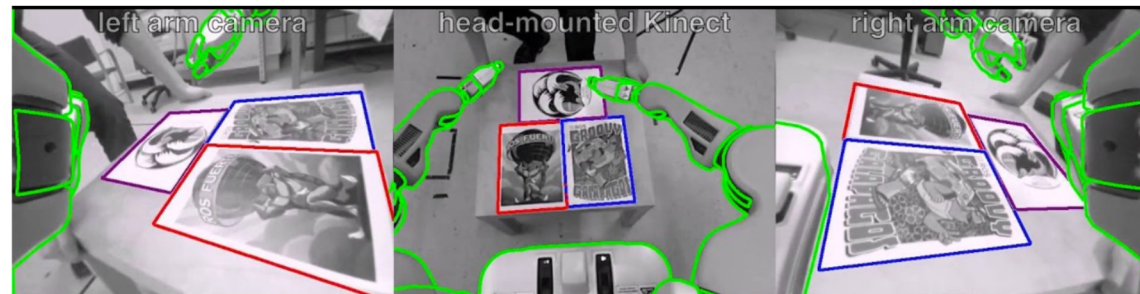
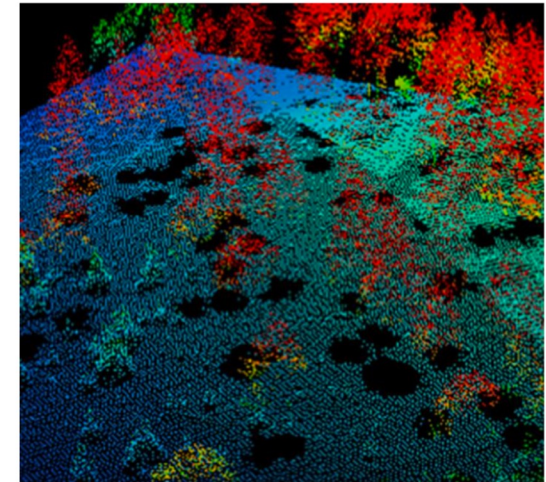
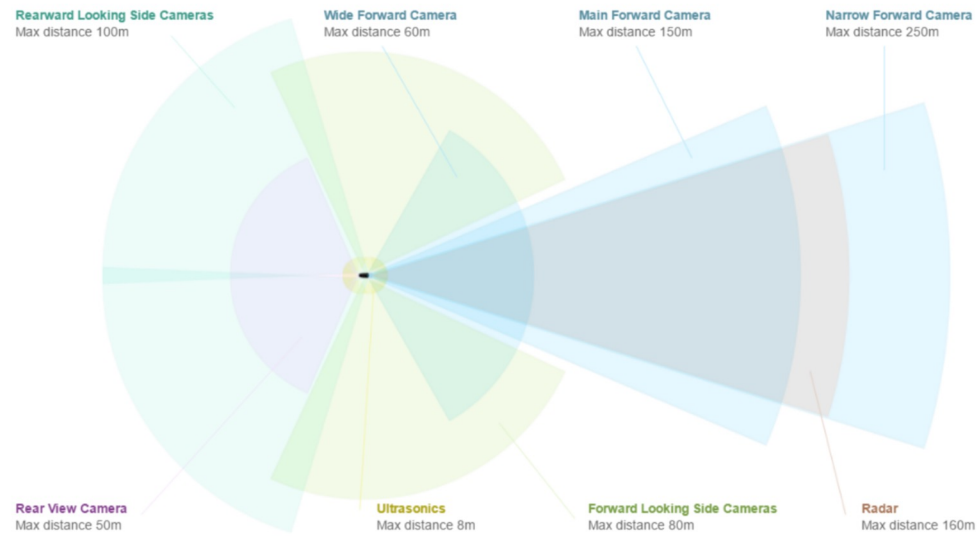
Politecnico
di Torino



Real-world states are complex and ambiguous



Observations are high-dimensional, multimodal and noisy



Where does reward come from in the real world?

The **specification of the reward** function is not that easy, e.g.,:

- Folding a towel
- Cooking a dinner
- Making a user happy



Designing controllers is a hard engineering problem

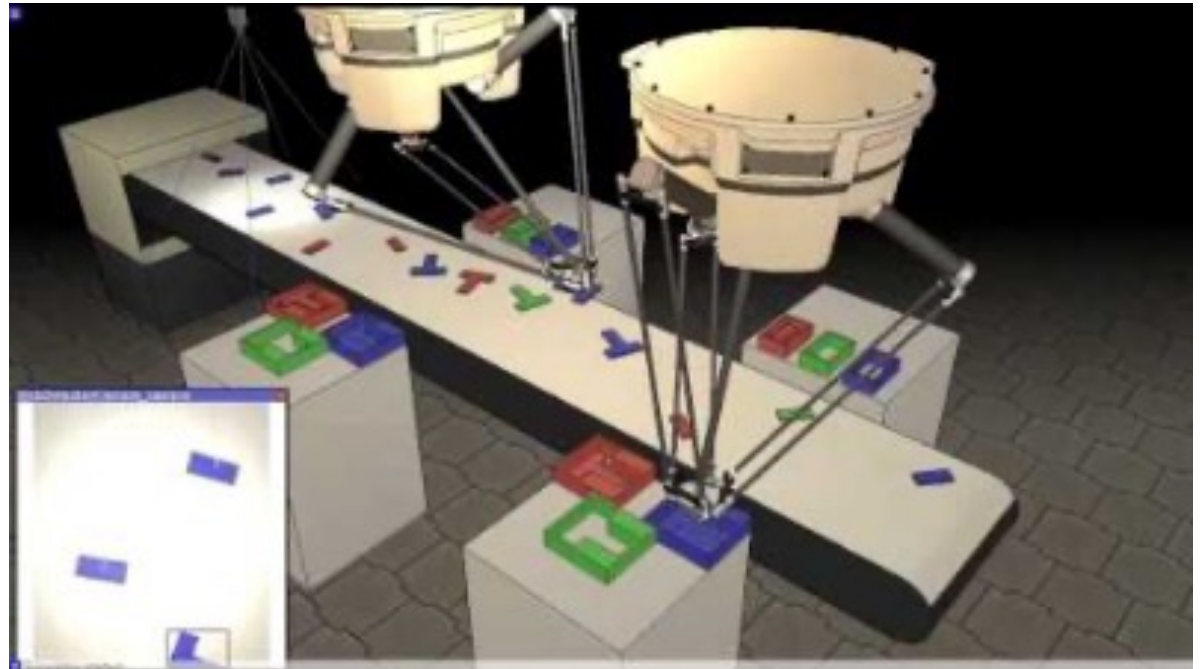
- Requires **good understanding of the system**
- Does not scale well to **high-dimensional systems**
- **Robots break** and degrade all the time



The need-of-data issue

Sampling data in real scenarios is very costly

Is simulated data the answer?



Pros of simulation

- Cheap, fast, and scalable
- Safe
- Labeled
- Not beholden to real-world probability distributions

Easy access to labels and rewards



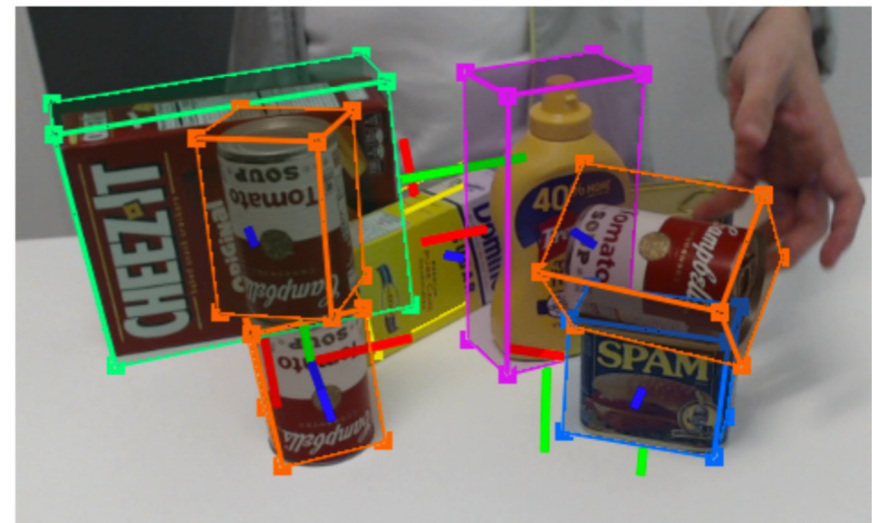
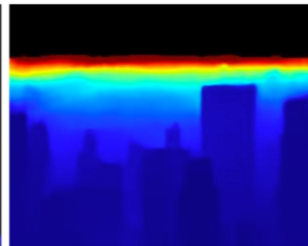
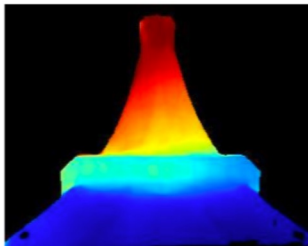
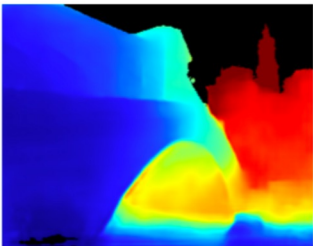
Rialto Bridge, Venice



Eiffel Tower, Paris



Central Park, NYC



Rare events

By definition, **rare events** are under-represented in the dataset

May be an **issue for robustness** of models

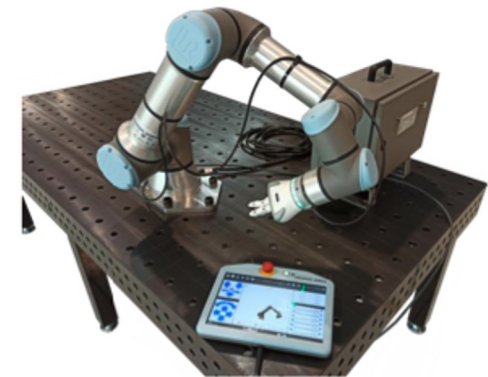
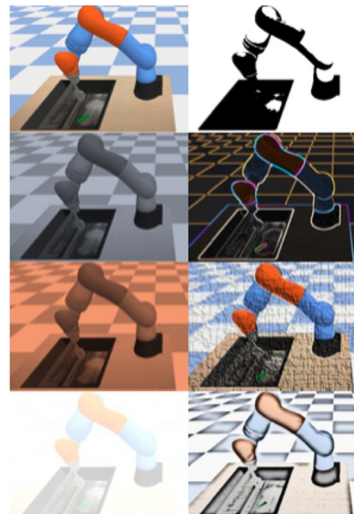
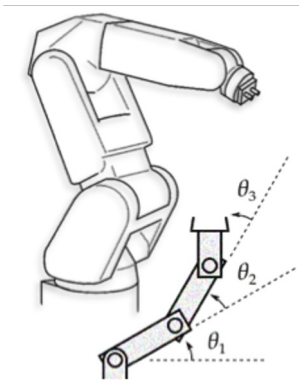
Synthesized data could
be a solution



But again, no free lunch

“There is a real danger (in fact, a near certainty) that programs which work well on simulated robots will completely fail on real robots because of the differences in real world sensing and actuation - it is very hard to simulate the actual dynamics of the real world.”

Artificial Life and Real Robots [Rodney Brooks, 1992]

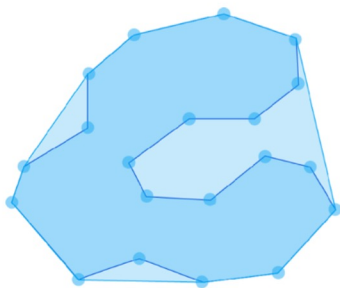


Cons of simulation

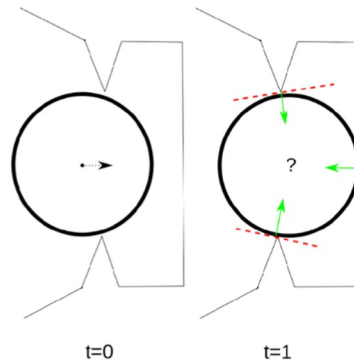
1. It is **hard** to accurately & efficiently **model sensors & physical systems**

E.g., physics simulators make big **assumptions** to run faster

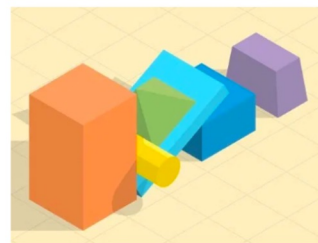
Convex objects



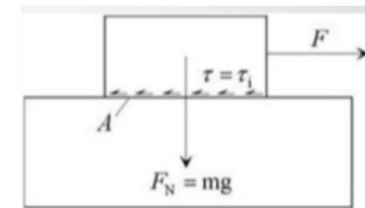
Discrete time



Rigid bodies



Coulomb friction



Etc

Cons of simulation

1. It is **hard** to accurately & efficiently **model sensors & physical systems**

The more accurate is the model the more data are required...
and more data makes the **simulation expensive and complex**



Ref: Toward Understanding Stories From Videos [Fidler et al, 2016]

Robot Learning

Photorealistic sensor
simulation

Jungle Book:
30M render hours
19 hours per frame
800 artist-years of effort¹⁶

Cons of simulation

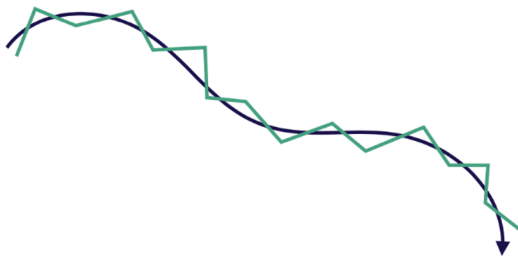
1. It is **hard** to accurately & efficiently **model sensors & physical systems**
1. **Small modeling errors lead to large control errors**

Cons of simulation

1. It is **hard** to accurately & efficiently **model sensors & physical systems**
1. **Small modeling errors lead to large control errors**

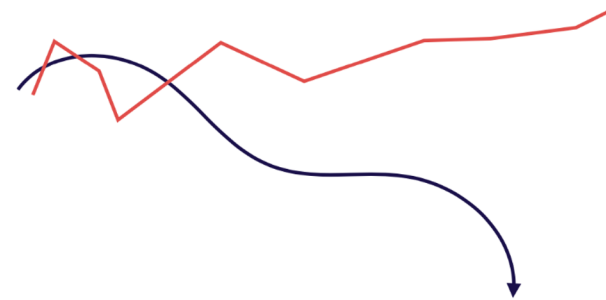
Errors often compounds!

What we hope happens



Uncorrelated errors

What actually happens

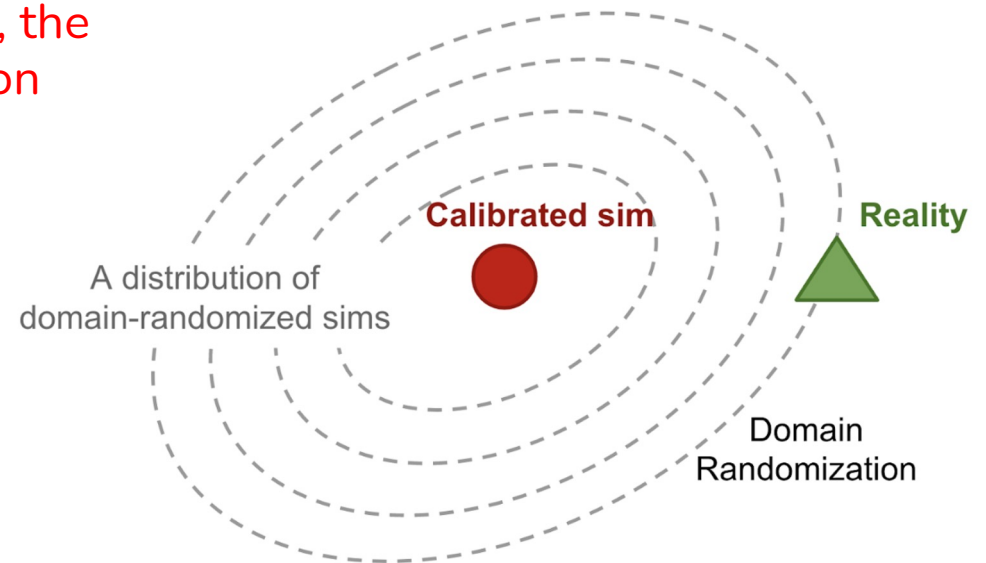
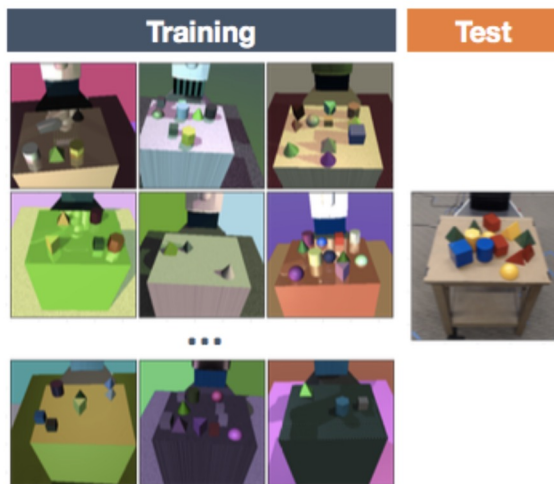


Compounding errors

Reducing the sim-to-real gap

Domain Randomization (DR)

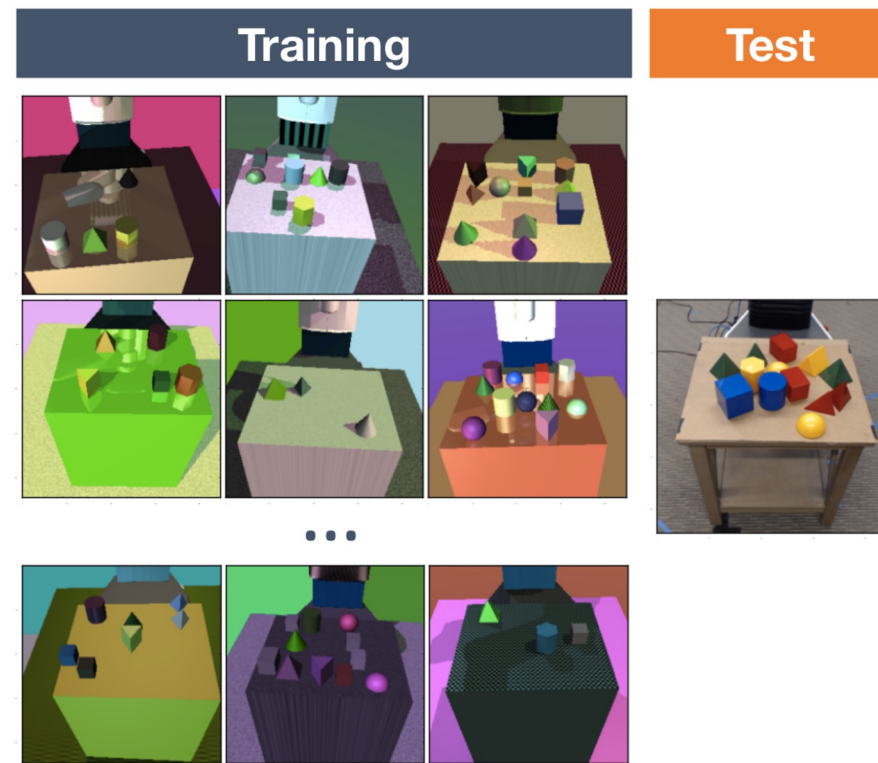
If the model sees enough simulated variations, the real world may just look like the next simulation



How does Domain Randomization work?

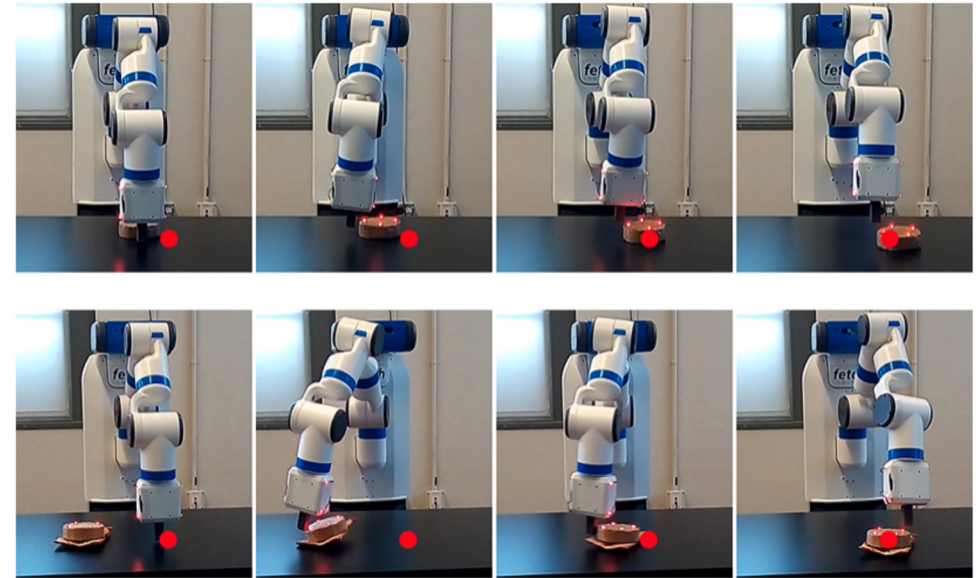
- **DR for appearance:** randomize textures, material properties, lighting conditions, camera positions, distractor objects
- **DR for dynamics:** randomize physical parameters in the simulator such as masses, friction coefficients, damping coefficients

Domain Randomization for appearance



Domain Randomization for dynamics

- **Standard RL:** train a feedforward neural network policy in a single best environment
- For each rollout, sample a different set of dynamics parameters



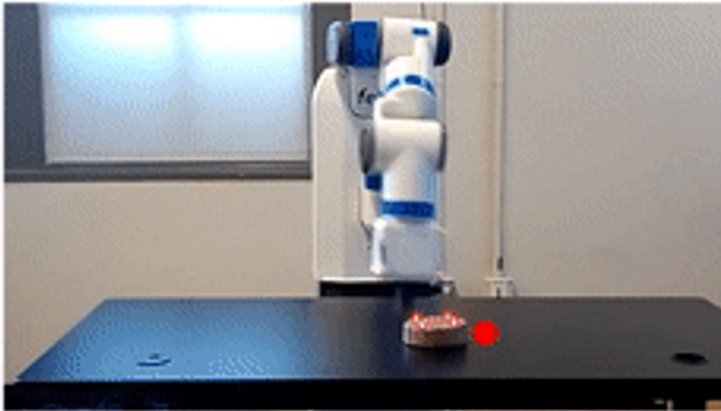
LSTM policy deployed on the Fetch arm.

Bottom: the contact dynamics of the puck was modified by attaching a packet of chips to the bottom

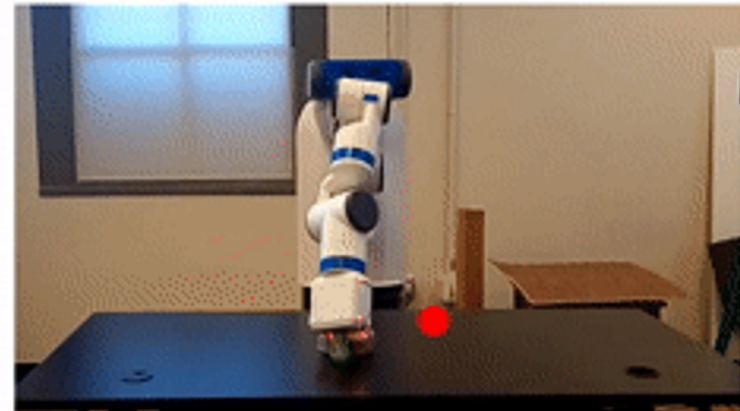
Parameter	Range
Link Mass	$[0.25, 4] \times \text{default mass of each link}$
Joint Damping	$[0.2, 20] \times \text{default damping of each joint}$
Puck Mass	$[0.1, 0.4] \text{ kg}$
Puck Friction	$[0.1, 5]$
Puck Damping	$[0.01, 0.2] \text{ N s/m}$
Table Height	$[0.73, 0.77] \text{ m}$
Controller Gains	$[0.5, 2] \times \text{default gains}$
Action Timestep λ	$[125, 1000] \text{ s}^{-1}$

Domain Randomization for dynamics

Comparisons



with DR

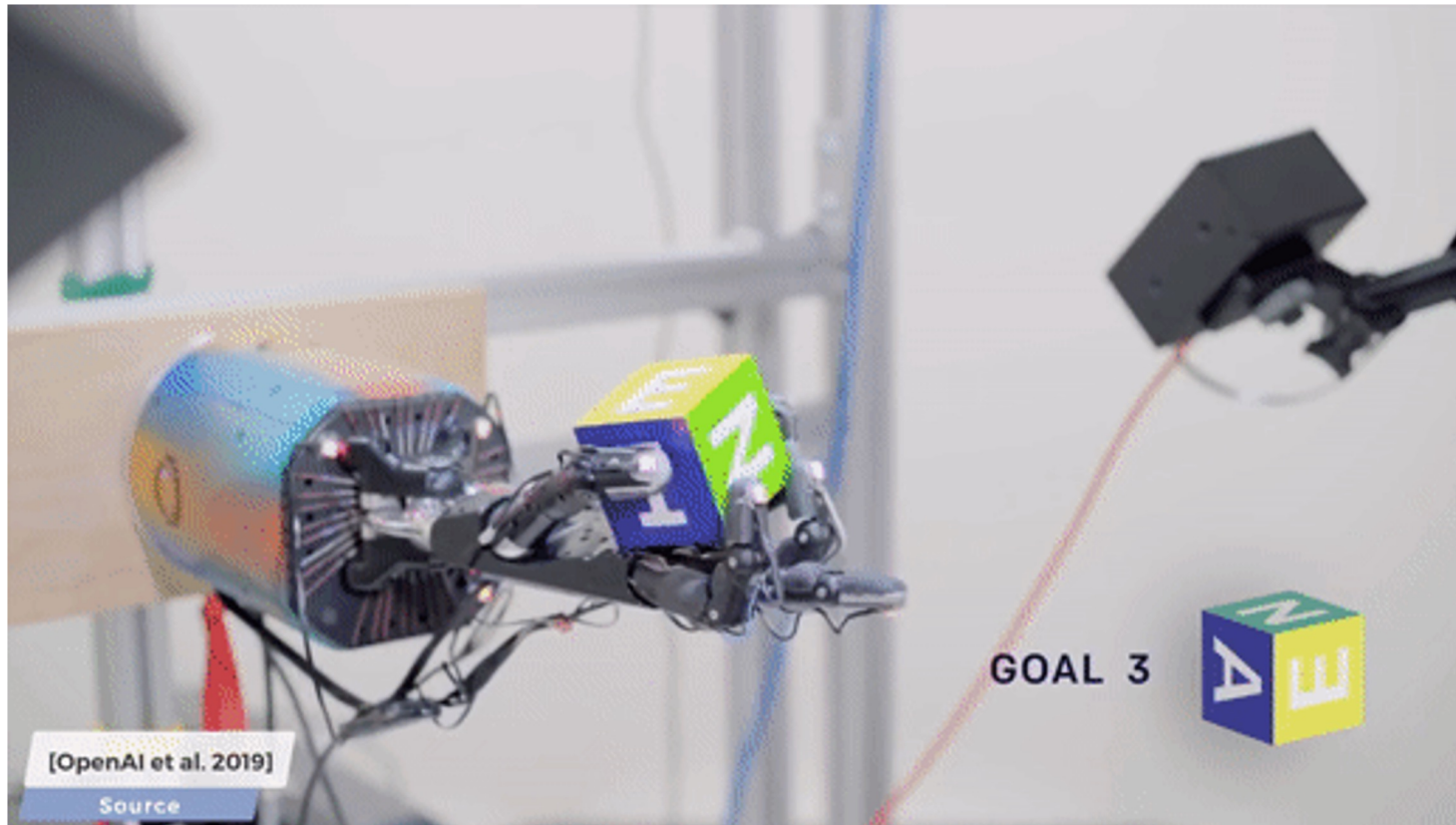


no DR



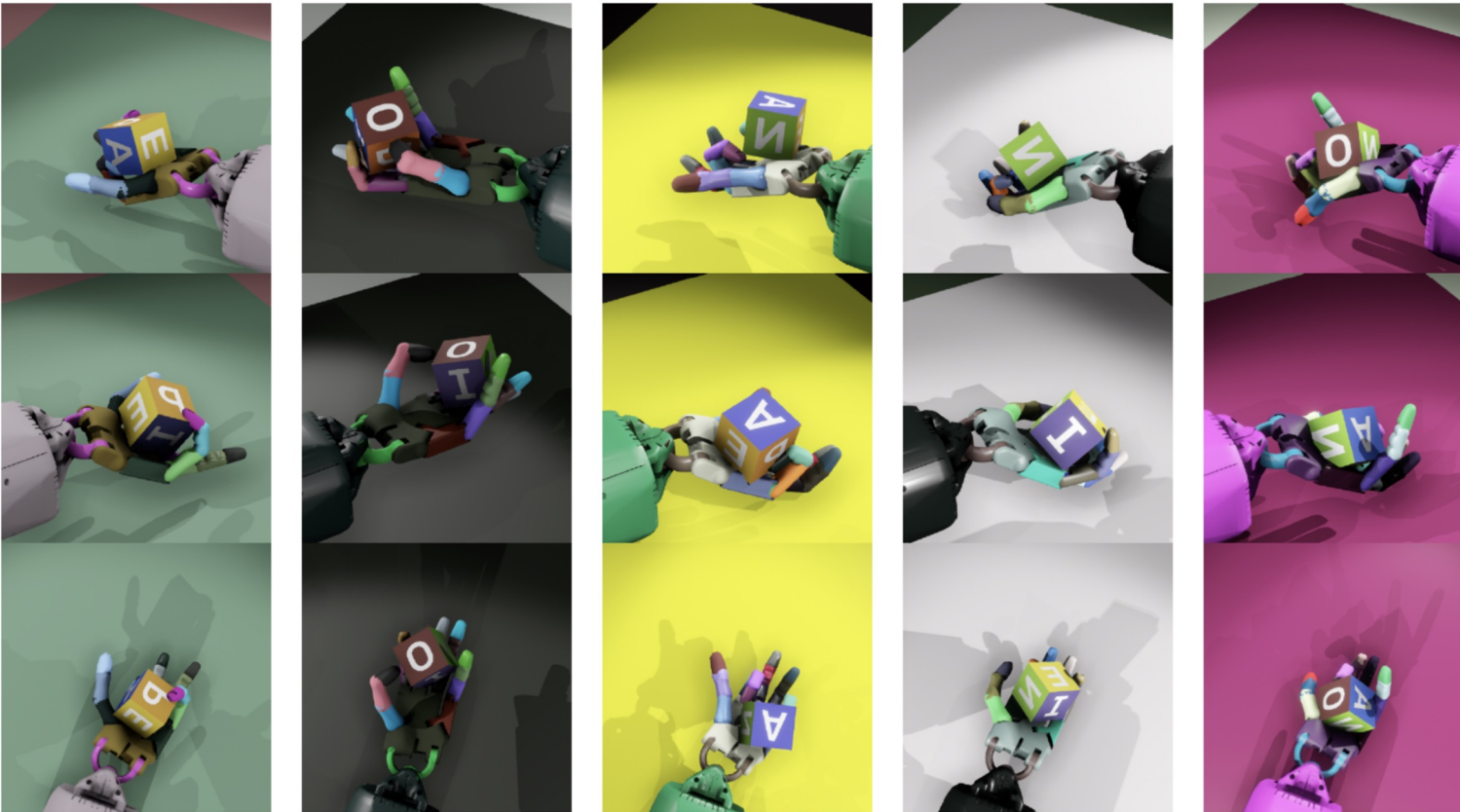
Politecnico
di Torino

In-hand manipulation



Ref: Learning Dextrous In-Hand Manipulation [OpenAI et al, 2018]

How does it work?



Example training
images used for
learning to
estimate the pose
of the block

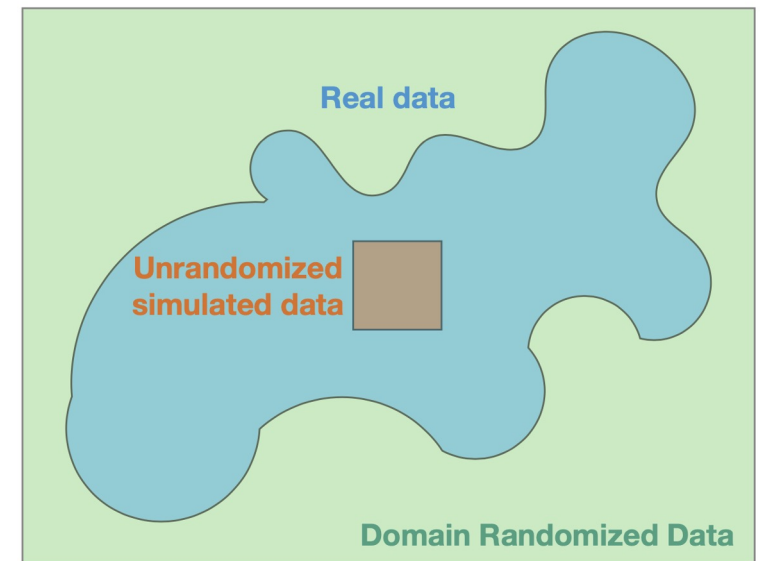
*Ref: Learning Dextrous In-Hand
Manipulation [OpenAI et al, 2018]*

What is randomized?

- Physical parameters
- Correlated and uncorrelated noise applied to policy inputs
- Sensor dropout
- Physics discretization timestep
- Random forces applied to the object

Why does domain randomization work?

- **Intuition 1:** training data comes from a **covering distribution**
- **Intuition 2:** domain randomization is a way of specifying to the model **what to ignore**
- **Intuition 3:** domain randomization is **meta-learning**



Challenges in applying domain randomization

- Building simulations is **manual** and **time consuming**
- Choose what parameters to **randomize** requires judgement
- **Randomizing parameters as much as possible may not be optimal**

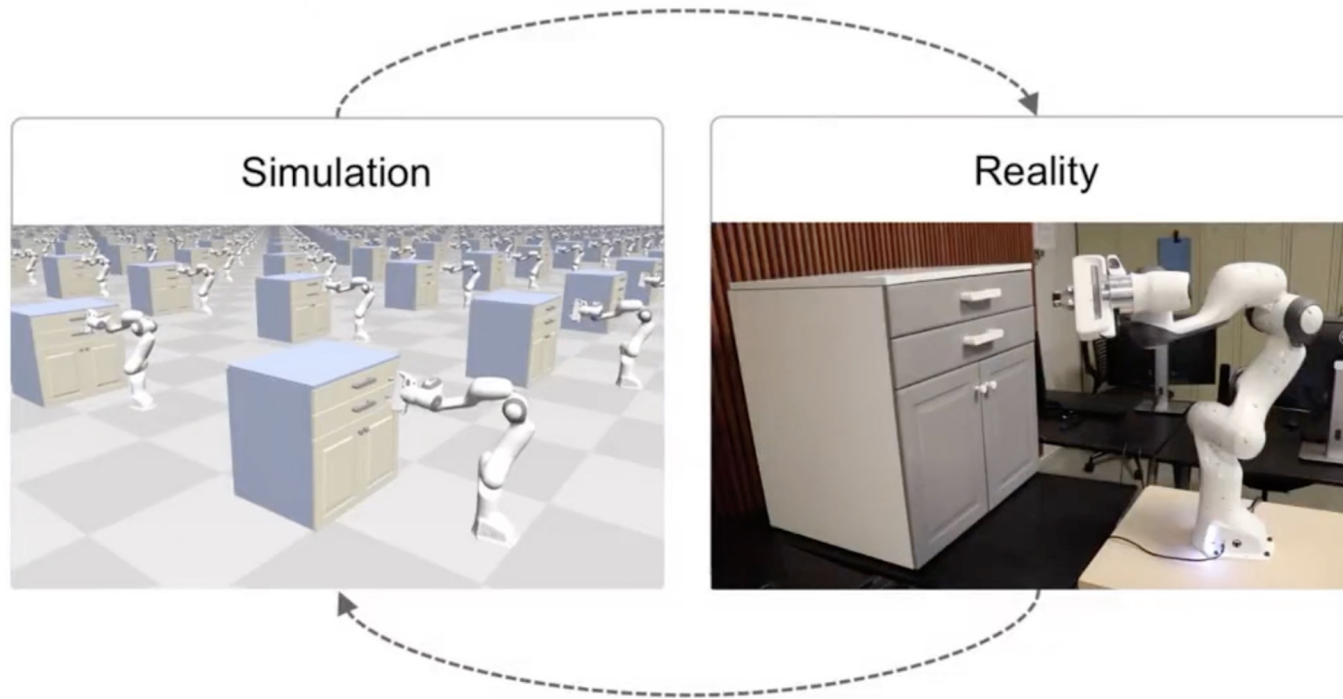
Extensions of domain randomization

- **Match the simulator to real data**
- **Use real data** to improve task performance
- Allow the model to perform well on a wider range of simulations



Politecnico
di Torino

SimOpt



Close the sim-to-real transfer loop:
Train policies on randomized simulated scenarios
Update simulation distribution to improve policy transfer

Ref: Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience [Chebotar et al, 2018]

Automatic Domain Randomization (ADR)

Intuition

- More randomization = better transfer, **given the same performance in sim**
- But wide randomization ranges lead to poor performance
- ADR: **automatically create a curriculum of expanding randomization ranges**
- How? Widen the distribution if performance is good near the boundary of the range

Ref: Solving Rubik's Cube with a Robot Hand [OpenAI et al, 2019]

Automatic Domain Randomization (ADR)

How parameter ranges are updated

- **Standard domain randomization:** randomizations are fixed.
- **ADR:** keep track of performance when each parameter is on boundary. Periodically, **if average performance is good, widen range. If bad, narrow range.**

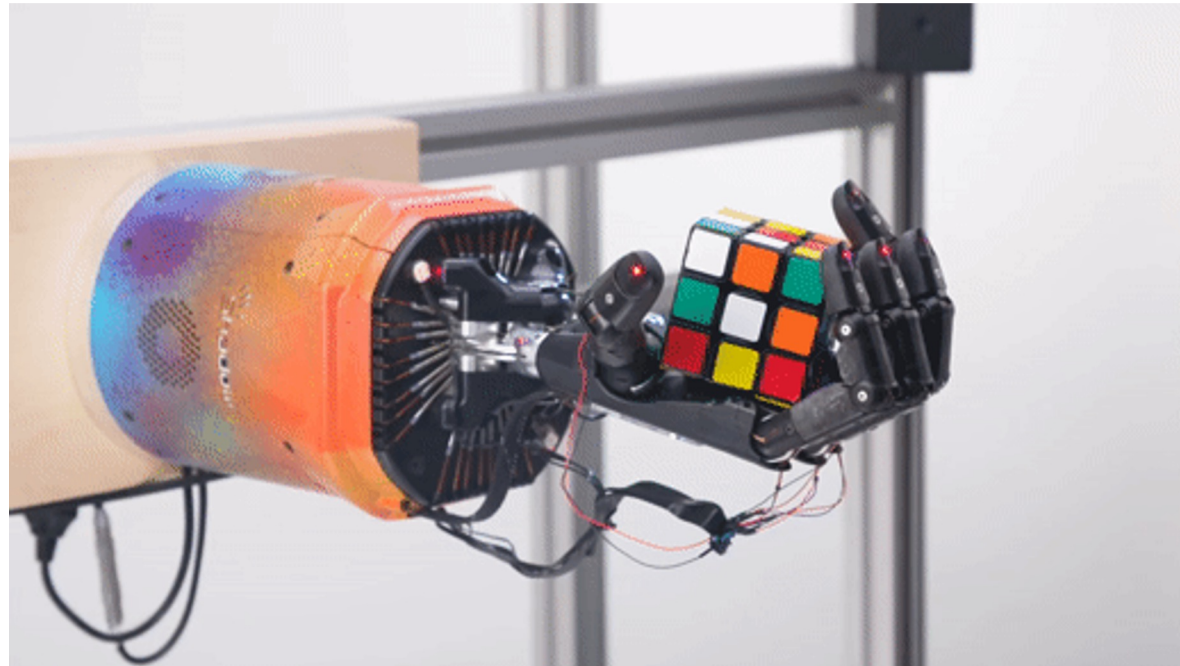
Ref: Solving Rubik's Cube with a Robot Hand [OpenAI et al, 2019]



Politecnico
di Torino



Automatic Domain Randomization (ADR) - Results



Ref: Solving Rubik's Cube with a Robot Hand [OpenAI et al, 2019]

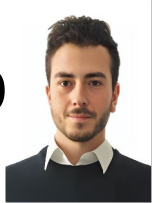
A few desiderata for Sim2Real

- **More / better domain randomization tools**
- **More accurate / scalable simulations**
- The next generation of Sim2Real techniques:
 - **More / better automatic domain generalization**
 - Domain randomization, domain adaptation, and model-based RL coverage
- The next generation of Sim2Real use cases:
 - **Synthetic data for edge cases**
 - **Synthetic data for reducing bias**
 - **Synthetic data on a hard, real-world dataset**

What are we doing

Just an example

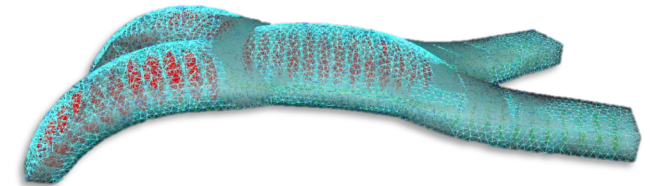
Closing the Reality Gap for Soft Robot Closed-loop Control with Domain Randomization



Presented at IROS 2023

MSc thesis work by Andrea Protopapa

Master Degree in Computer Engineering - *Politecnico di Torino, Turin, Italy*

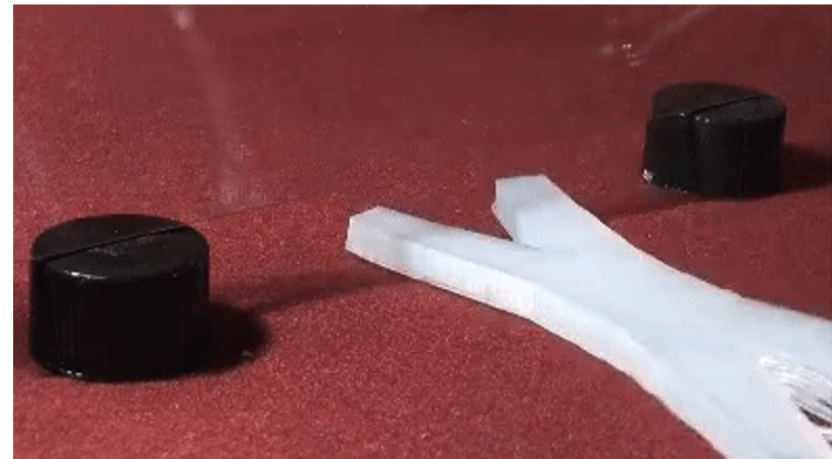


Soft Robotics

- New and rapidly growing research area in robotics community
- Use of soft and **elastic** materials (e.g., silicone rubbers)
- Inspiration by biological bodies, motor capabilities of animals
- Intrinsic **flexibility** and **adaptability** to the surrounding environment



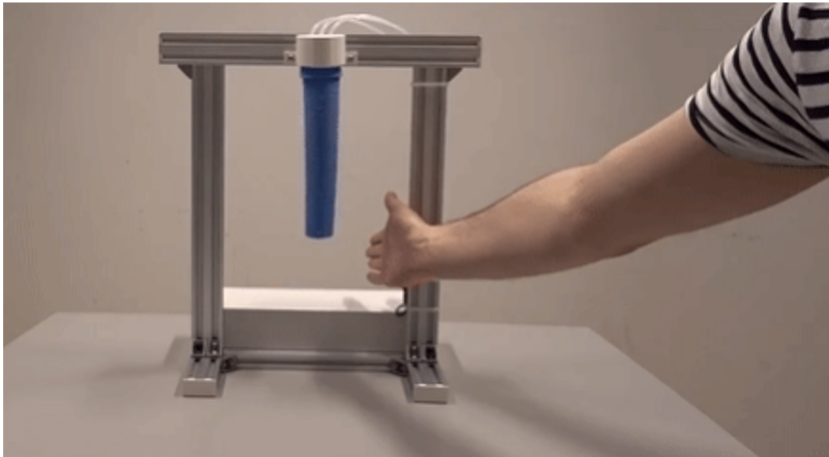
E. Coevoet et al. "Soft robots locomotion and manipulation control using FEM simulation and quadratic programming." 2019



R. Shepherd et al. "Multigait soft robot." 2011

Soft Robotics

- Many applications with the necessity of **agility, maneuverability, dexterity, safety**
 - Healthcare
 - Exploration and rescue operations
 - Handling delicate objects
 - Safe interaction with humans



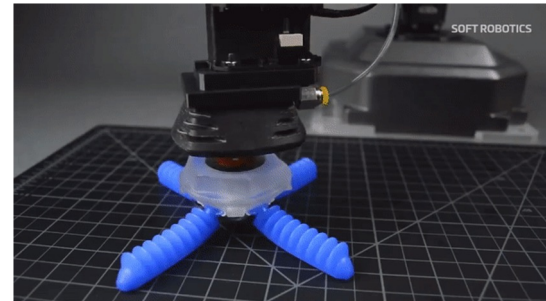
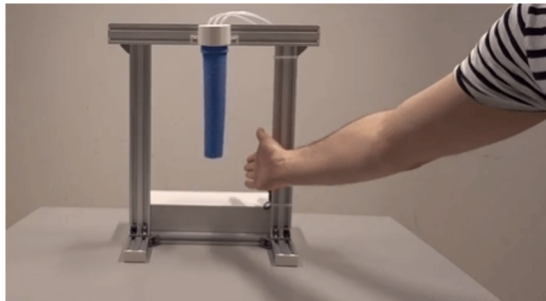
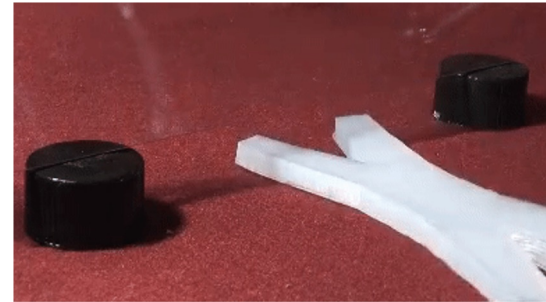
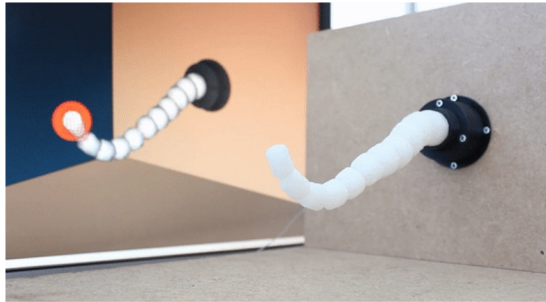
J. Jørgensen. "Interaction with soft robotic tentacles." 2018



Soft Robotics Inc., Soft Robotics, <https://www.softroboticsinc.com/> (2017)

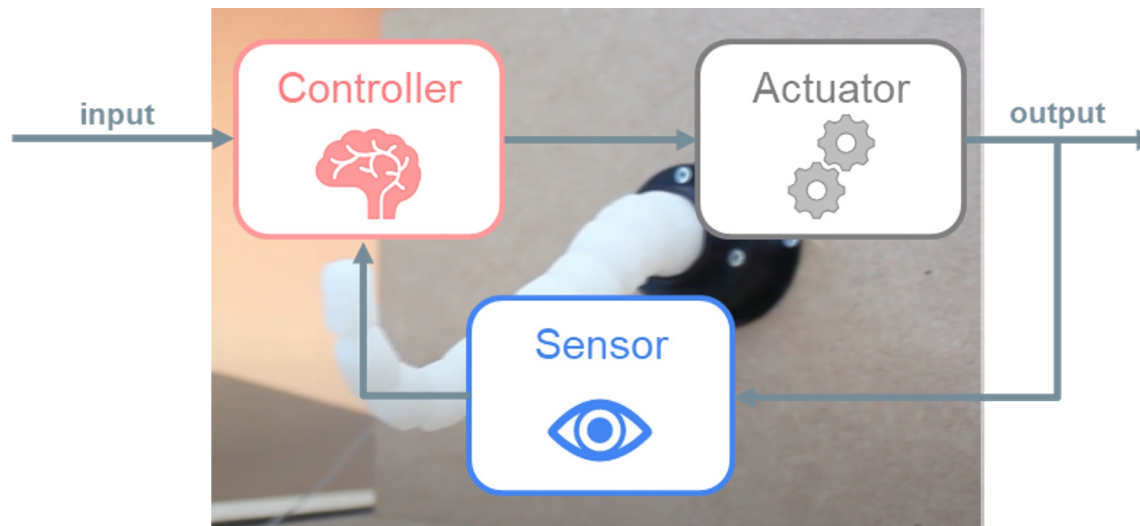
Soft Robotics

- Mathematically modeling and controlling soft materials is **challenging**
 - Highly non-linear dynamics, infinite-Degrees-of-Freedom systems



Closed-loop control for Soft Robots

- Sensors provide **feedback** about the deformation state of the actuators and the surrounding environment



Closed-loop control for Soft Robots

- Sensors provide **feedback** about the deformation state of the actuators and the surrounding environment

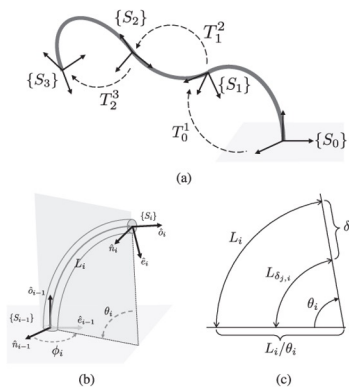


Closed-loop control for Soft Robots

- Sensors provide **feedback** about the deformation state of the actuators and the surrounding environment
- **Model-based** approaches use analytical models to obtain the controller



Model-based

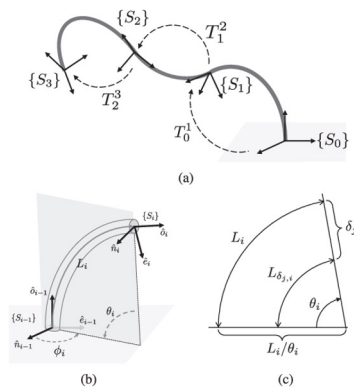


Closed-loop control for Soft Robots

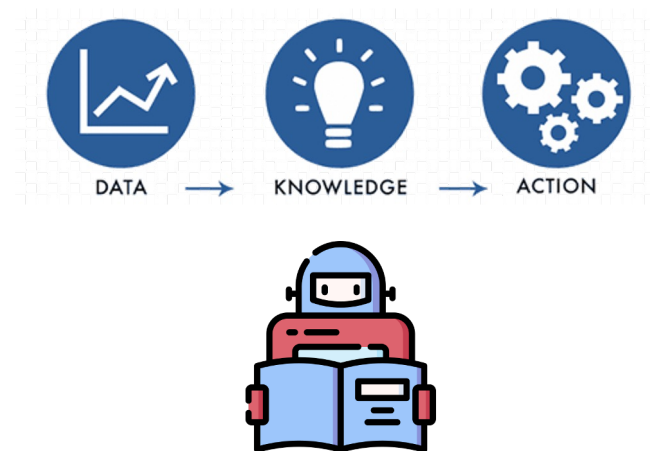
- Sensors provide **feedback** about the deformation state of the actuators and the surrounding environment
- **Model-based** approaches use analytical models to obtain the controller
- **Data-driven** techniques learn from empirical data, overcoming the mathematical limitations
 - **Reinforcement Learning (RL)** has shown remarkable success stories



Model-based

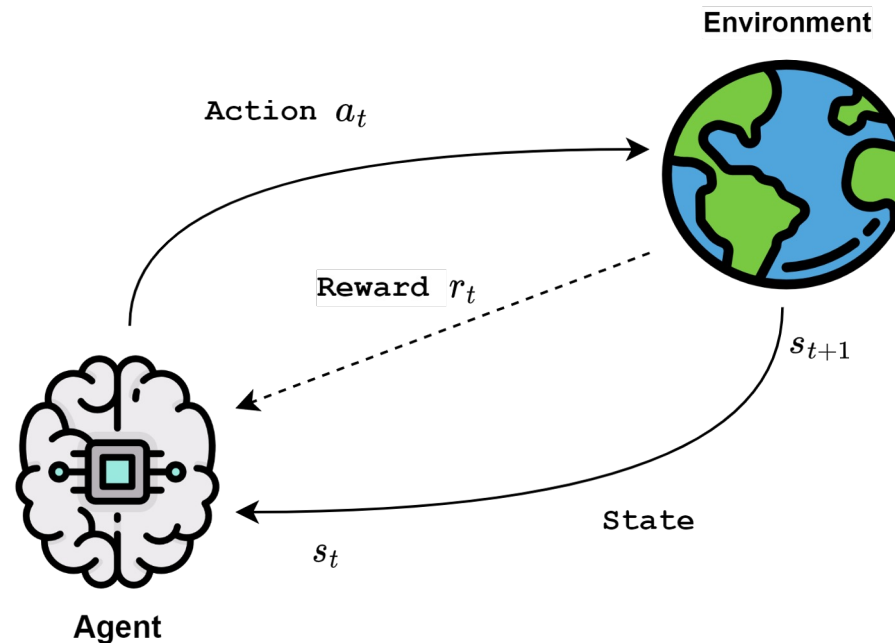


Data-driven



Reinforcement Learning

- Area of Machine Learning, where the agent (i.e., the robot) directly learns how through **interactions** and observations with its **environment**, based solely on **experience**
- May raise **safety concerns** in a real environment



Reinforcement Learning for Soft Robots

- Learning in the **real world**, collecting real-world interaction data

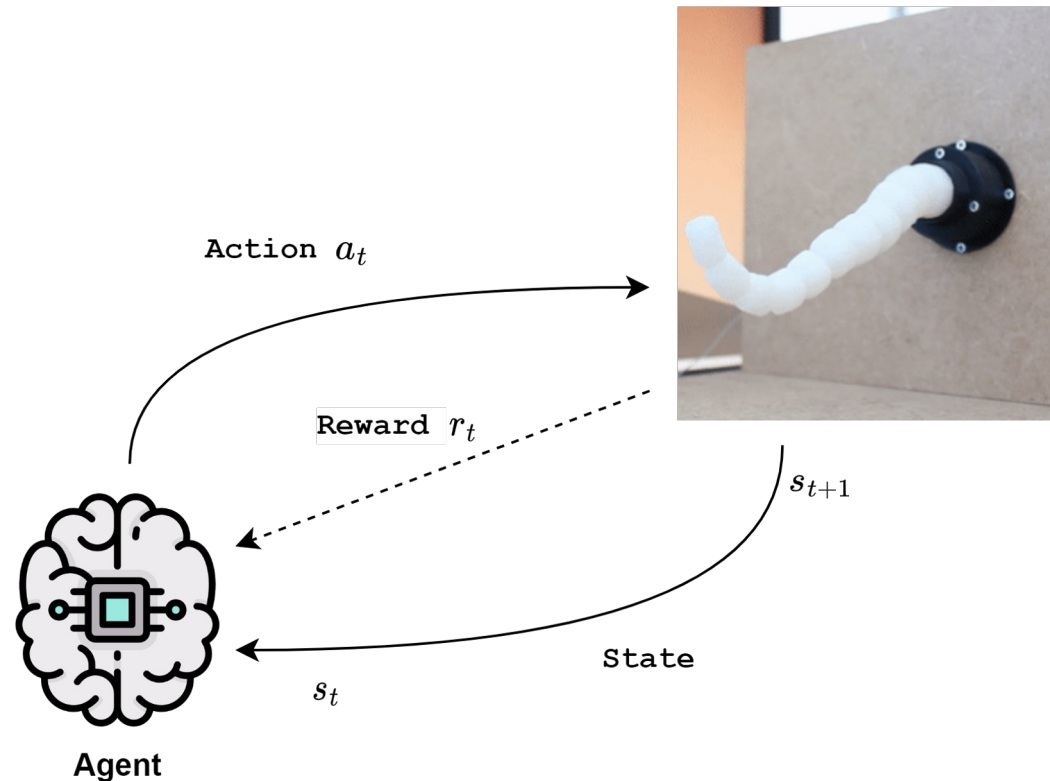
✓ Realistic observations

✗ Data inefficient

✗ Time-consuming and noisy operation

✗ Uncontrolled exploration

✗ Real-world reward computation



Reinforcement Learning for Soft Robots

- Learning in a **simulation environment**, collecting data from a simulated physical engine

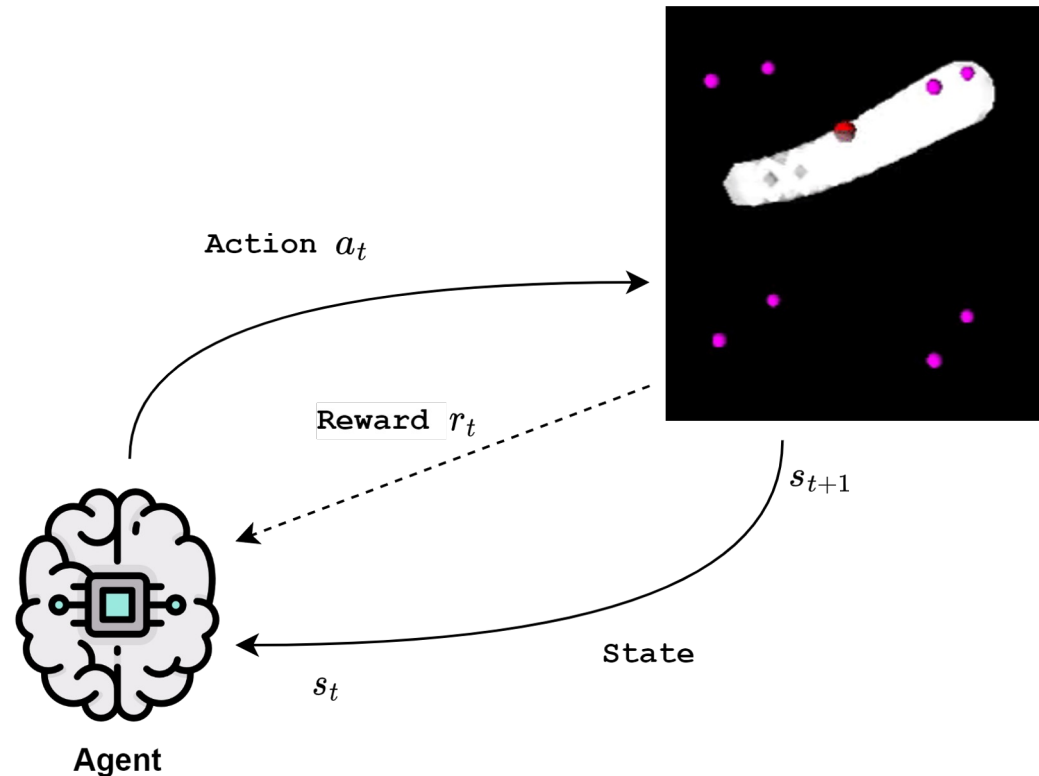
✓ Cheaper and faster data collection

✓ Safe exploration

✓ Various scenarios on parallel environments

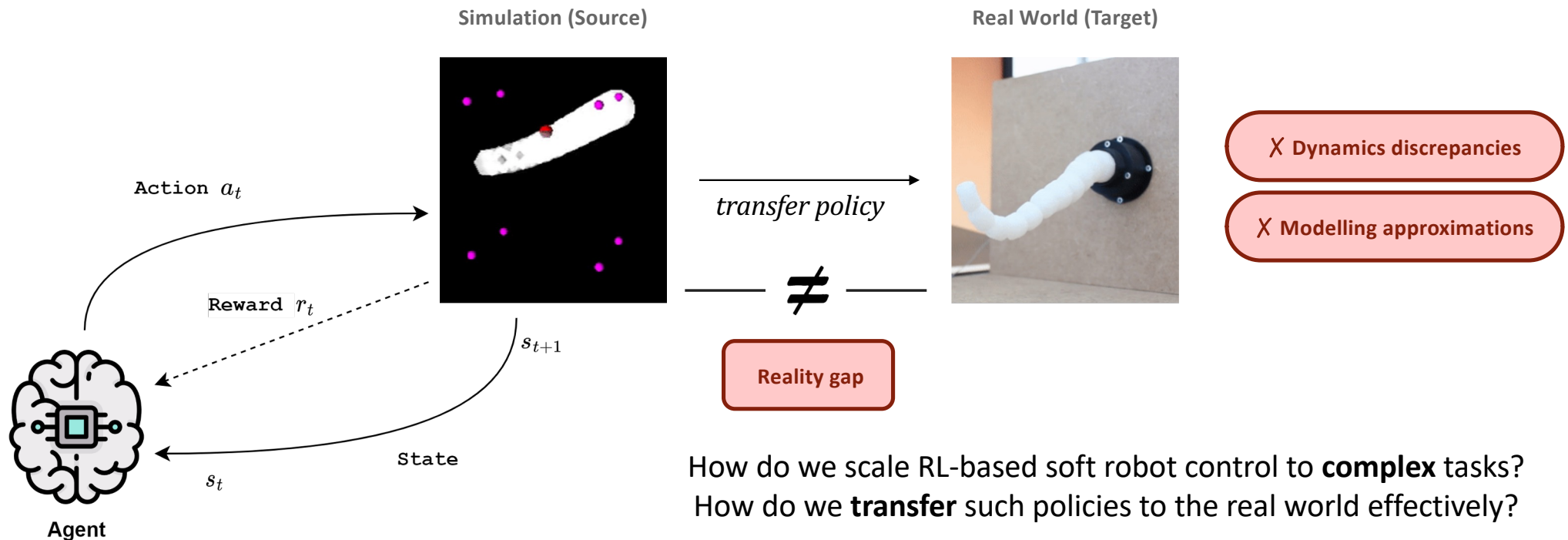
✓ Easy reward computation

✗ Reality gap



Sim-to-real Transfer

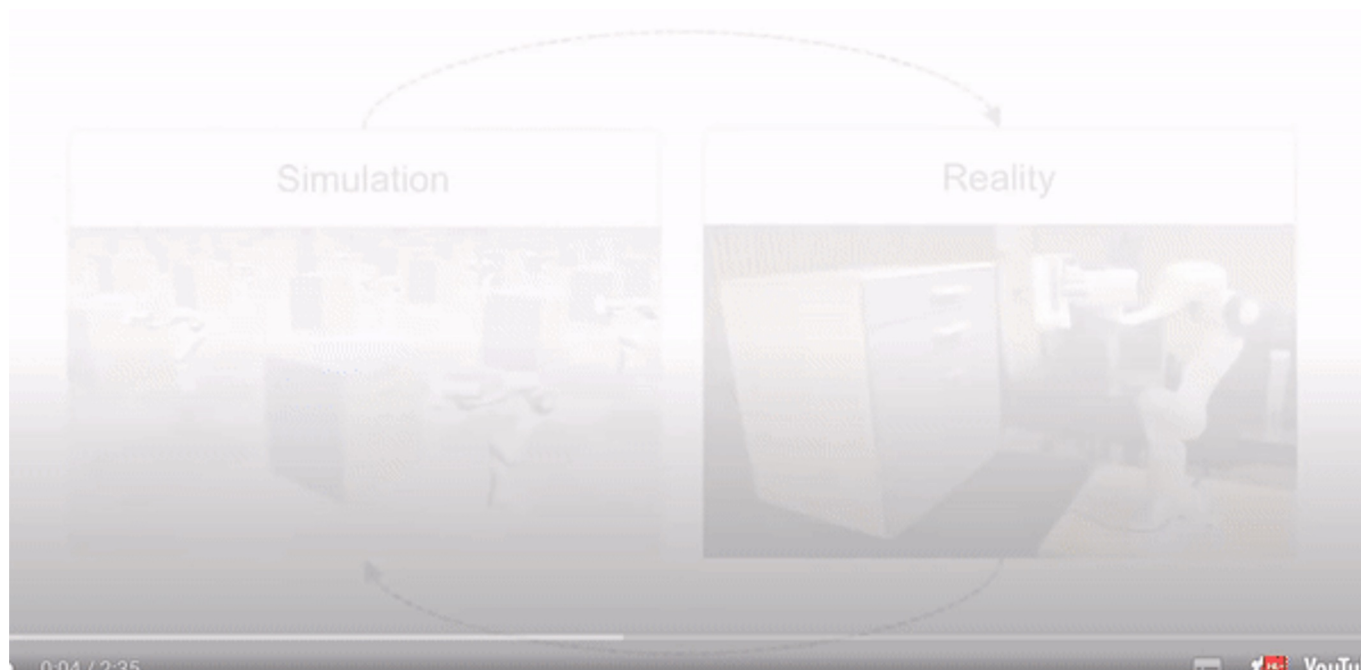
- Transfer a learned policy from **simulation training** to a **real prototype evaluation**
- **Reality-gap**: discrepancies between simulated model and real world



How do we scale RL-based soft robot control to **complex** tasks?
How do we **transfer** such policies to the real world effectively?

Domain Randomization

- Training on a **wide range** of simulated environments with **randomized** physical dynamics parameters
- Hope of ensure learning an optimal behaviour **robust to variations** in real-world



Y. Chebotar et al. "Closing the sim-to-real loop: Adapting simulation randomization with real world experience." 2019

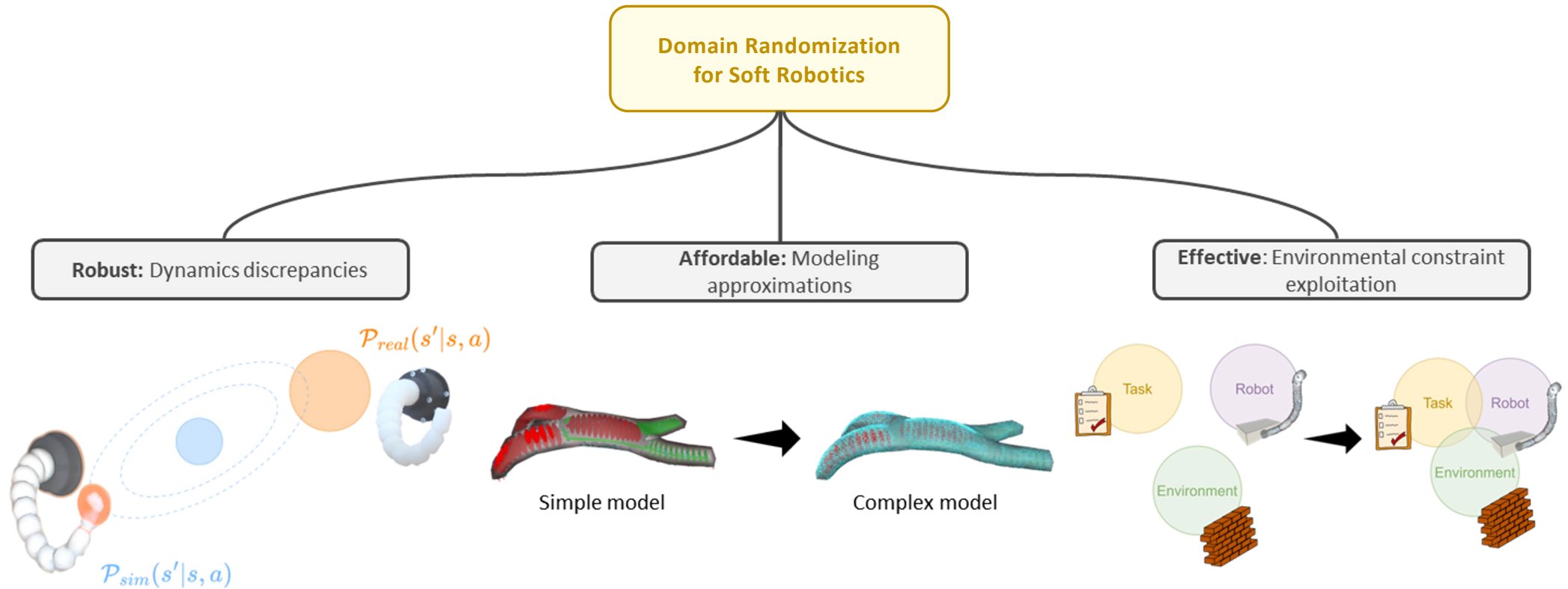
Domain Randomization

- Successfully adopted in rigid robotics, improving transferability from simulation to reality
- **Largely unexplored** field in **soft robotics**



Y. Chebotar et al. "Closing the sim-to-real loop: Adapting simulation randomization with real world experience." 2019

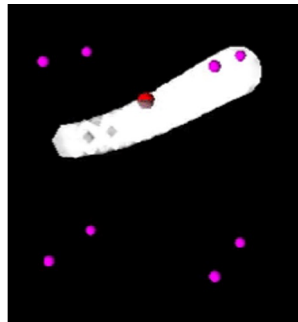
Experiments and Results



Domain Randomization in Soft Robotics

- **Randomize dynamic parameters** of the robot and the environment at the start of each training episode according to some **predefined distribution** (e.g. gaussian or uniform distributions)

Simulation (Source)



Training in simulation on
randomized dynamics

$$mass \sim \mathcal{N}(\mu_1, \sigma_1^2)$$

$$Poisson's\ ratio \sim \mathcal{N}(\mu_2, \sigma_2^2)$$

$$Young's\ modulus \sim \mathcal{N}(\mu_3, \sigma_3^2)$$

...

Real World (Target)

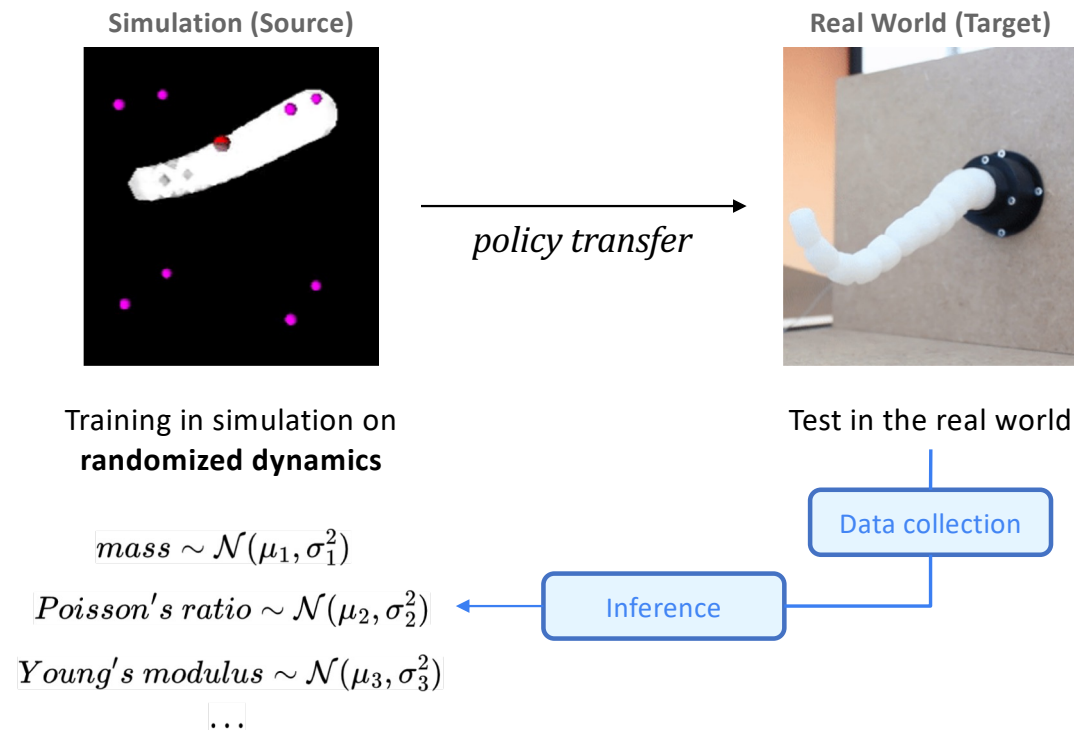


Test in the real world

policy transfer

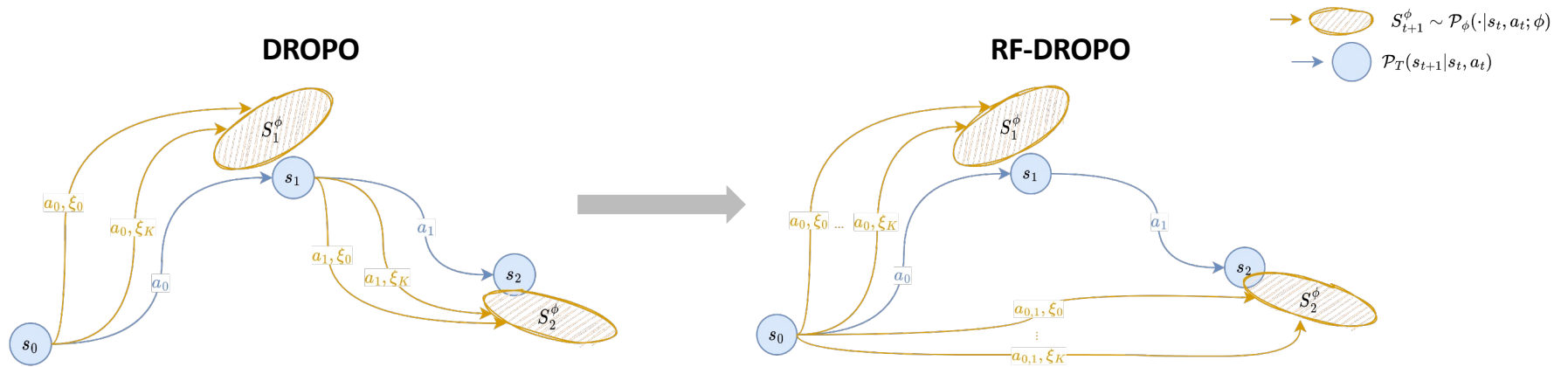
Adaptive Domain Randomization in Soft Robotics

- **Inference** methods to **estimate** the exact DR distributions of parameters used for sampling the simulator



RF-DROPO

- *Reset-Free* DROPO: a novel extension to the state-of-the-art ADR method DROPO [1] to manage **partially observable environments**, such as deformable and soft manipulators
 - DROPO adopts a **maximum-likelihood approach** to replay real-world data in simulation with **intermediate state-resetting**
 - RF-DROPO relaxes the assumption, assuming of **knowing only the initial configuration** of the environment and **executing all preceding actions**



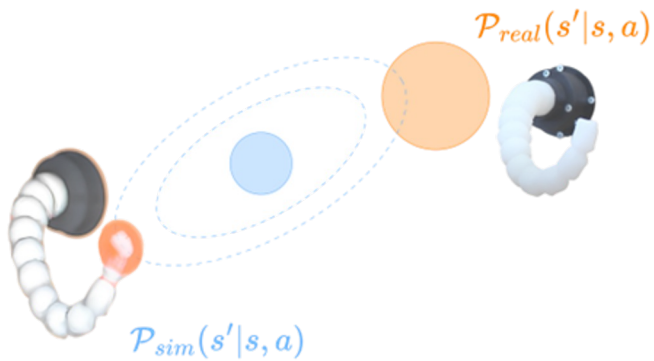
DR for dynamics discrepancies

Domain Randomization
for Soft Robotics

Robust: Dynamics discrepancies

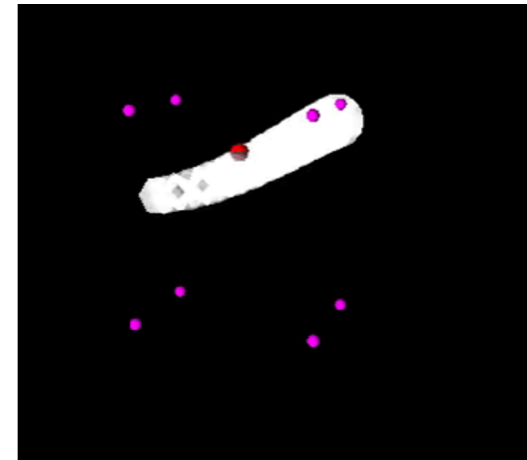
Affordable: Modeling
approximations

Effective: Environmental constraint
exploitation



DR for dynamics discrepancies: TrunkReach

- **Trunk** robot: a silicone **cable-driven** continuum deformable manipulator
- Simple reaching task, no contact-rich
- **Unknown dynamics parameters:** trunk mass, Poisson's ratio, Young's modulus



Task
Reach target location
(red dot)

DR for dynamics discrepancies: TrunkReach

Dynamics parameters inference results

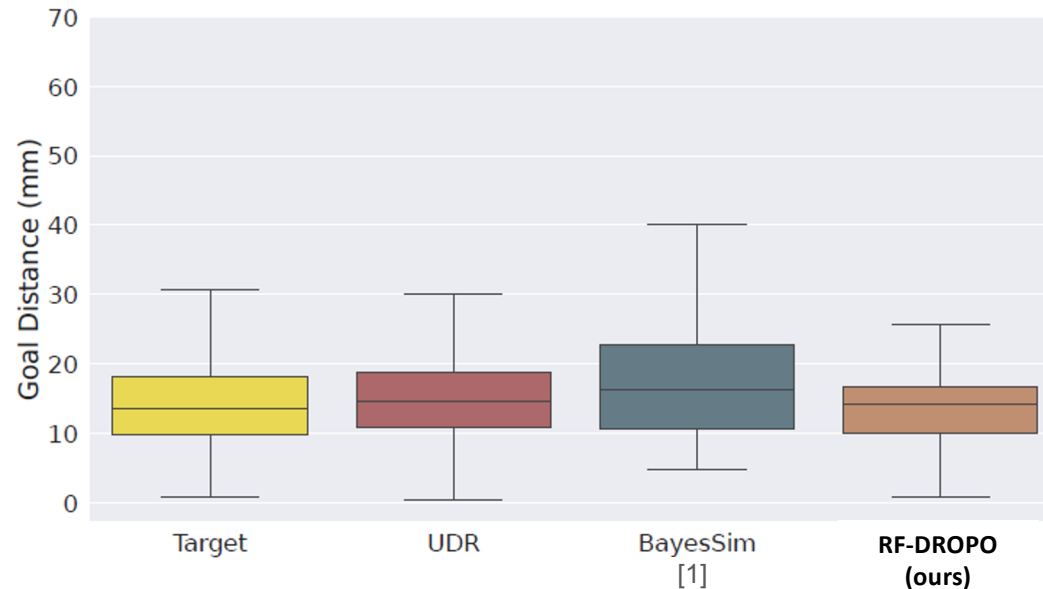
		Trunk Mass [kg]	Poisson's Ratio	Young's Modulus [kg/(mm · s ²)]
Target		0.42	0.45	4500
Search space	[min, max]	[0.005, 1]	[0.4, 0.5]	[2000, 7000]
BayesSim[1]	(μ, σ)	0.32, 1.28E-01	0.49, 2.30E-03	1914.25, 7.31E+06
RF-DROPO	(μ, σ)	0.64, 1.00E-05	0.45, 1.00E-05	6878.01, 2.77e-02

Task Reach target location (red dot)
Trunk mass, Poisson's ratio, Young's modulus randomized
Target data is collected to infer dynamics parameters prior to policy training

RF-DROPO **successfully converges** to the target Poisson's Ratio value,
while Trunk Mass and Young's Modulus are more difficult to infer

DR for dynamics discrepancies: TrunkReach

Policy transfer – quantitative evaluation



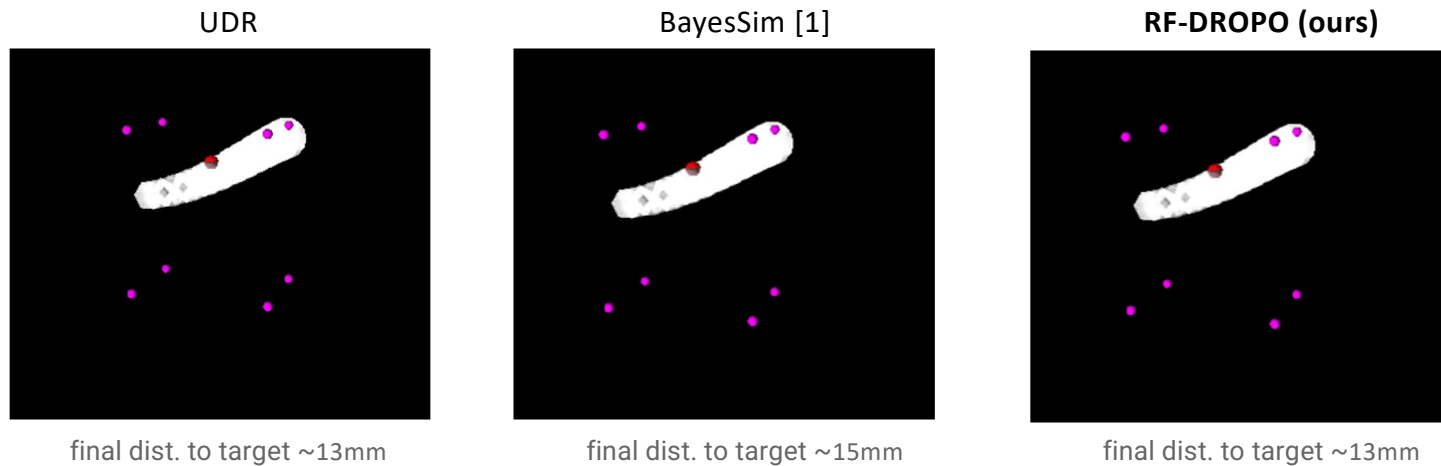
- Task
Reach target location
(red dot)
- Trunk mass, Poisson's
ratio, Young's modulus
randomized
- Target data is collected
to infer dynamics
parameters prior to
policy training

Simple reaching tasks may be solved effectively also **using conventional DR approaches**, even in presence of significant dynamics discrepancies between source and target domains

[1] F. Ramos et al. "BayesSim: adaptive domain randomization via probabilistic inference for robotics simulators." 2021

DR for dynamics discrepancies: TrunkReach

Policy transfer – quantitative evaluation



Task
Reach target location
(red dot)

Trunk mass, Poisson's
ratio, Young's modulus
randomized

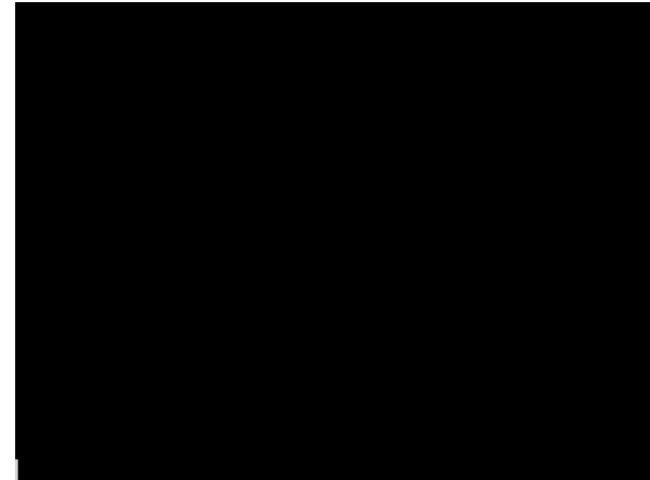
Target data is collected
to infer dynamics
parameters prior to
policy training

Simple reaching tasks may be solved effectively also **using conventional DR approaches**, even in presence of significant dynamics discrepancies between source and target domains

[1] F. Ramos et al. "BayesSim: adaptive domain randomization via probabilistic inference for robotics simulators." 2021

DR for dynamics discrepancies: TrunkPush

- **Trunk** robot: a silicone **cable-driven** continuum deformable manipulator
- Contact-rich task: **highly non-linear dynamics**
- **Unknown dynamics parameters**: trunk mass, cube mass, friction coefficient, Young's modulus, Poisson's ratio



Task
Push the object to a target location
(green dot)

DR for dynamics discrepancies: TrunkPush

Dynamics parameters inference results

		Cube Mass [kg]	Friction Coefficient	Trunk Mass [kg]	Poisson's Ratio	Young's Modulus [kg/(mm · s ²)]
Target		0.05	0.3	0.42	0.45	4500
Search space	[min, max]	[0.005, 1]	[0.01, 1]	[0.005, 1]	[0.4, 0.5]	[2000, 7000]
BayesSim [1]	(μ, σ)	0.37, 9.43E-02	0.37, 1.07E-01	0.69, 1.29E-01	0.42, 3.50E-03	1855.10, 8.17E+06
RF-DROPO	(μ, σ)	0.06, 4.40E-03	0.30, 1.51E-03	0.52, 3.24E-03	0.45, 4.50E-04	5557.07, 2.45E+00

Task
Push the object to
a target location
(green dot)

Trunk mass, cube mass,
friction coefficient,
Young's modulus
Poisson's ratio
randomized

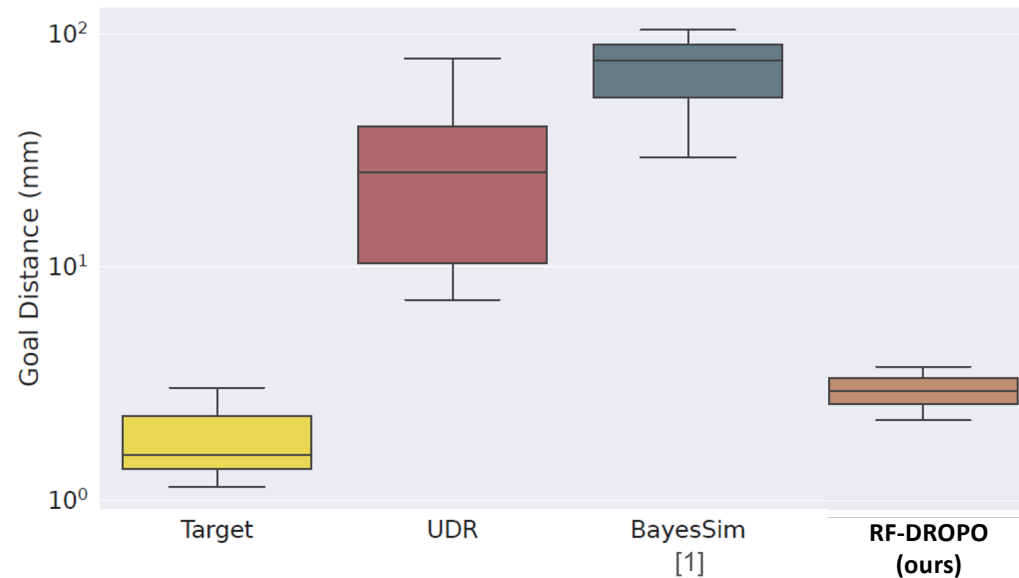
Target data is collected
to infer dynamics
parameters prior to
policy training

RF-DROPO **successfully converges** to the target Poisson's Ratio value, while Young's Modulus is more difficult to infer, obtaining generally accurate results compared to BayesSim

[1] F. Ramos et al. "BayesSim: adaptive domain randomization via probabilistic inference for robotics simulators." 2021

DR for dynamics discrepancies: TrunkPush

Policy transfer – quantitative evaluation



Task
Push the object to a target location (green dot)

Trunk mass, cube mass, friction coefficient, Young's modulus, Poisson's ratio randomized

Target data is collected to infer dynamics parameters prior to policy training

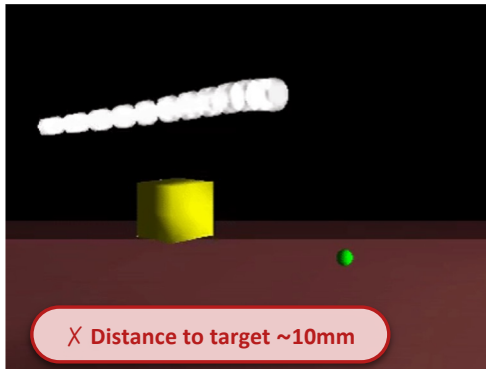
DR distributions estimated by RF-DROPO are used to train a **robust policy** that can **transfer well to the target domain** with nominal parameter values

[1] F. Ramos et al. "BayesSim: adaptive domain randomization via probabilistic inference for robotics simulators." 2021

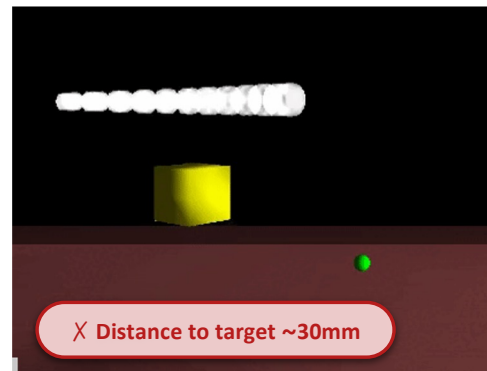
DR for dynamics discrepancies: TrunkPush

Policy transfer – qualitative evaluation

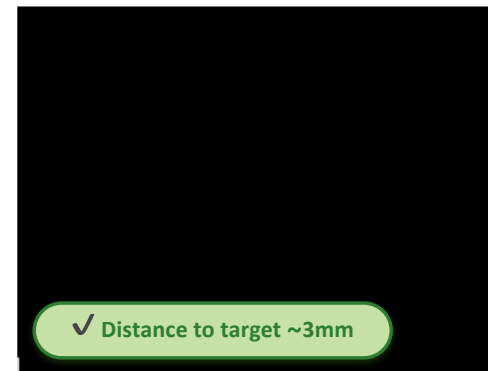
UDR



BayesSim [1]



RF-DROPO (ours)



Task

Push the object to
a target location
(green dot)

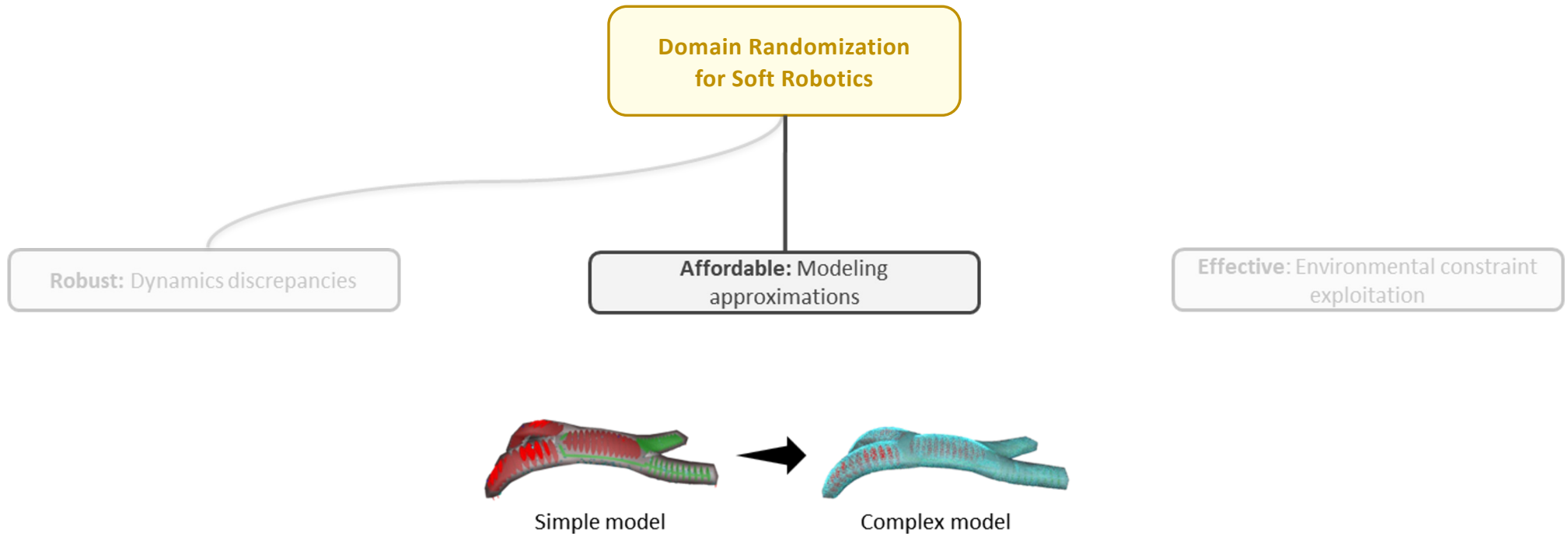
Trunk mass, cube mass,
friction coefficient,
Young's modulus
Poisson's ratio
randomized

Target data is collected
to infer dynamics
parameters prior to
policy training

DR distributions estimated by RF-DROPO are used to train a **robust policy** that can **transfer well to the target domain** with nominal parameter values

[1] F. Ramos et al. "BayesSim: adaptive domain randomization via probabilistic inference for robotics simulators." 2021

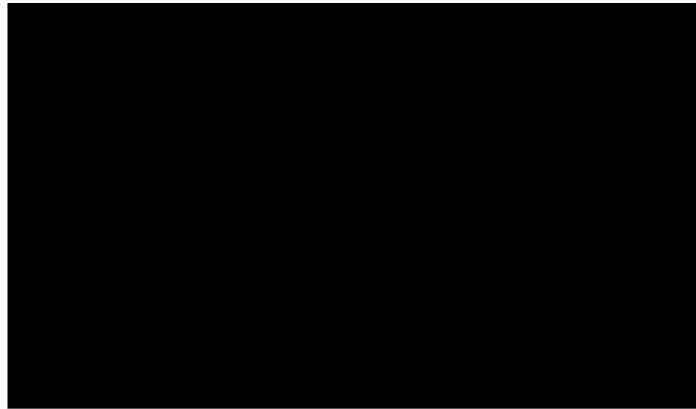
DR for complex modeling



DR for complex modeling: Multigait

- **Multigait** robot: highly complex **pneumatic-based** robot
- Locomotion task
- Transferring learned policy from the simple to the complex model

Task
Walk forward



Simplified model



Accurate model

Training time:
(500k timesteps, 24 CPUs)

~ 7 days

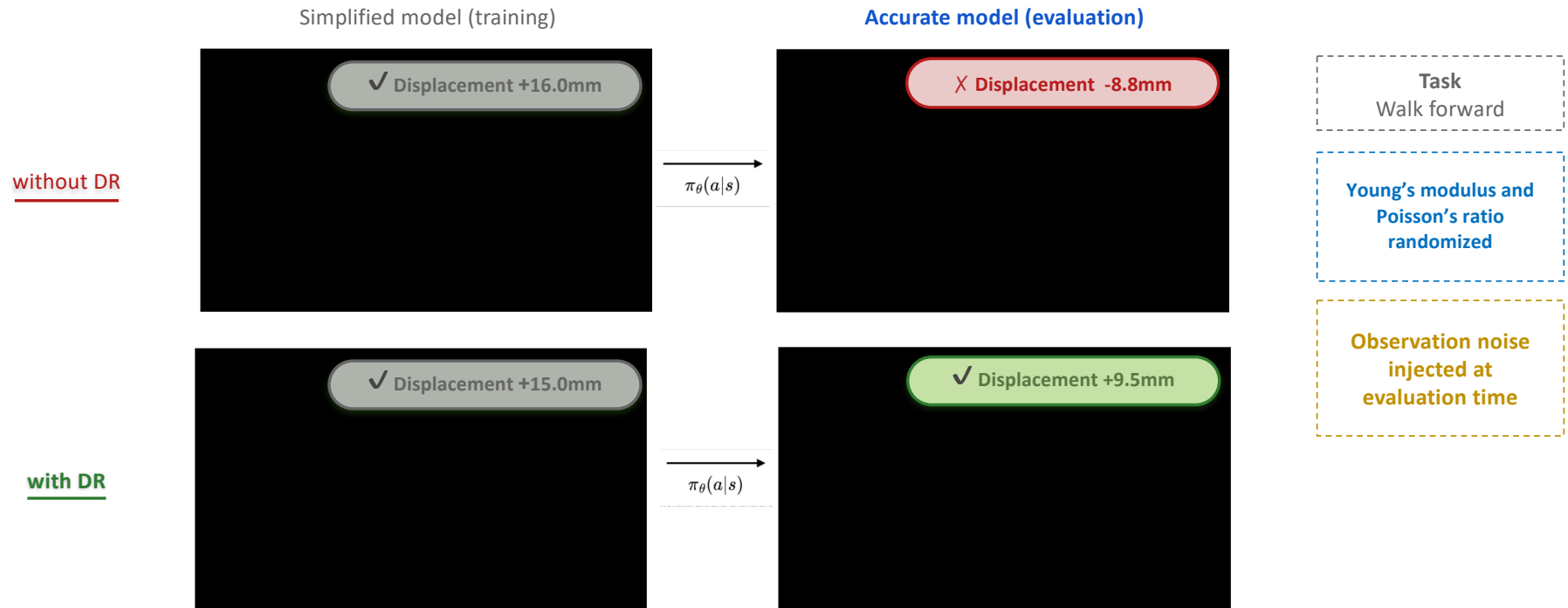
~ **55 days**

Test time:
(12 timesteps, 1 CPU)

1 min

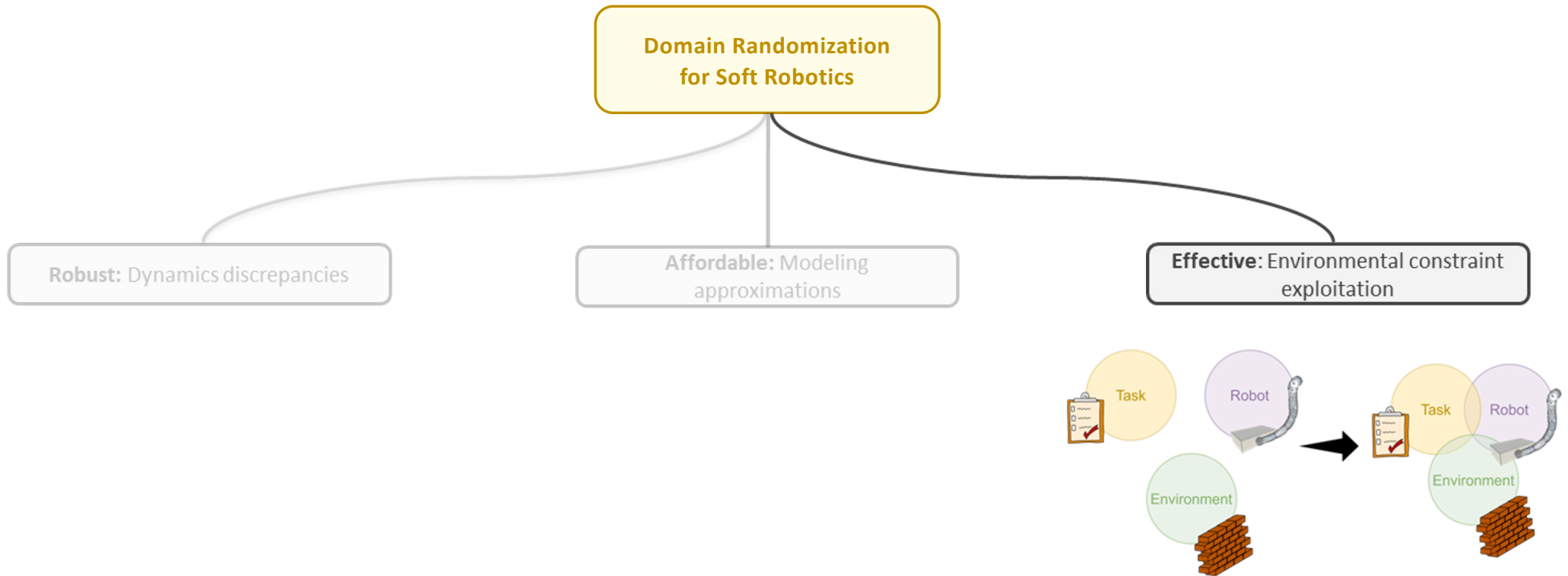
8 min

DR for complex modeling: Multigait



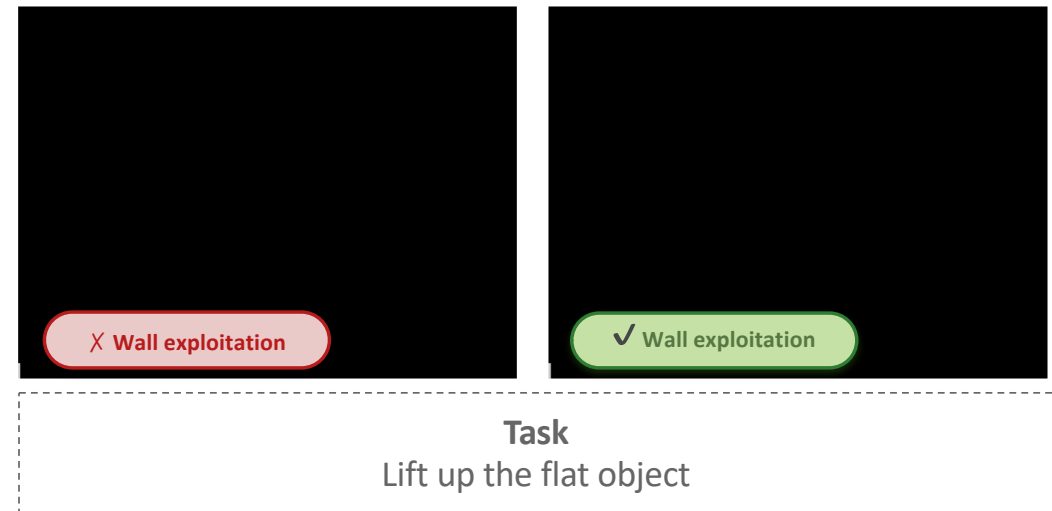
Domain Randomization allows policies to be trained **~8x faster on simpler models**, which well-transfer to more complex models while being **robust to noise**.

DR for Environmental Constraint Exploitation

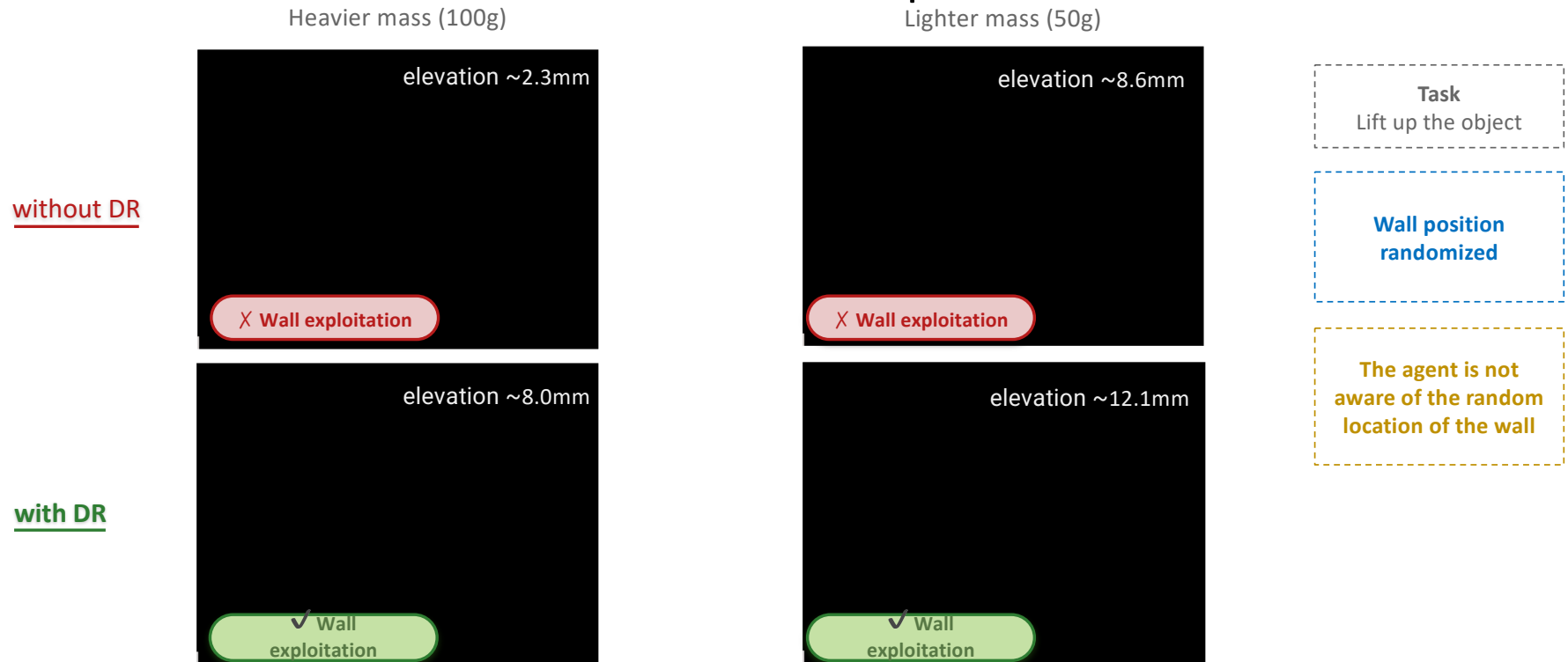


DR for Environmental Constraint Exploitation: TrunkLift

- **Trunk** robot: a silicone **cable-driven** continuum deformable manipulator
- Contact-rich task: **highly non-linear dynamics**
- Infinite-DoF system: **multiplicity** of potential solutions
- Deformable materials: **safely** comply with objects and environmental constraint

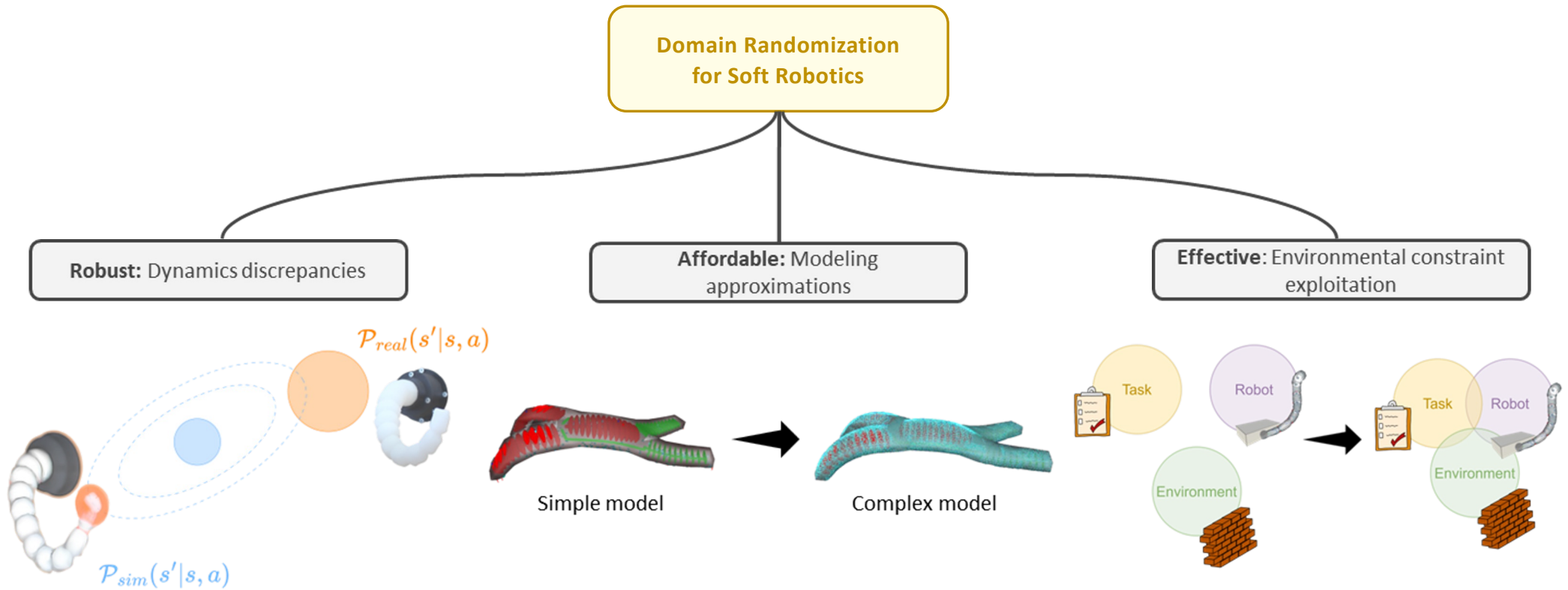


DR for Environmental Constraint Exploitation: TrunkLift



Randomizing the location of the wall at training time allows agents to discover **more effective strategies** to lift up the object **exploiting the environment**

Takeaways





Politecnico
di Torino



Future works



Investigate the transferability in the **real world**



Improve and enhance the capabilities of RF-DROPO.



Expand the methods to tasks with **high dimensional state-space**



Study even more challenging contact-rich manipulation tasks, such as **grasping**